

Designing a Backend Monitoring System:->

Introduction

When we start learning backend monitoring, we come across tools like Prometheus and Grafana. They are powerful, but as beginners, we find them a bit overwhelming. So instead of just using them, we want to understand how a monitoring system actually works by building a smaller one ourselves.

Our initial idea is pretty simple:

Backend → Monitoring Server → Database → Dashboard

We want to track things like API health, request count, latency, error rate, and basic server health such as CPU and memory usage. We also want alerts so that issues can be detected and debugged faster.

But as we start thinking about the design, the simple approach quickly raises more questions. How do we collect metrics without affecting the application? What happens when the number of requests grows? Should every metric go directly to the database?

So we start with the simplest design and evolve the architecture as these problems come up.

2. The Problem & Requirements

The main goal of the system is to get a better understanding of what is happening inside a backend application.

For every API request, the system tracks the endpoint, HTTP method, status code, response latency, whether it resulted in a server error, and the time at which the request completed.

Along with API metrics, it also collects basic server health metrics such as CPU and memory usage. These metrics can then be viewed over different time periods to understand how the system behaves.

The monitoring system should help answer questions like:

- * Which endpoints are slow?
- * Which endpoints are producing more errors?
- * How many requests are we getting?
- * How are latency and error rates changing over time?
- * Is an increase in API latency related to CPU or memory usage?
- * Can we get an alert when a metric crosses a threshold?

There are also a few important constraints.

****Low latency:**** Collecting metrics should add as little overhead as possible to the application.

****High request volume:**** The system should handle metrics generated by a large number of requests across different endpoints.

****Isolation:**** If the monitoring system becomes slow or unavailable, it should not affect the host application.

****Easy integration:**** Adding monitoring to an existing backend should require minimal changes, ideally just integrating an SDK.

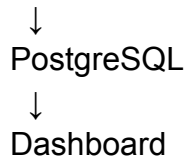
These requirements give us a starting point. Now let's look at the simplest architecture that can solve this problem.

3. The Simplest Design

We start with a very simple architecture.

Whenever the backend receives a request, we collect the required metrics and send them to the monitoring server through another HTTP request.





For example, when a request reaches `/api/orders`, we collect information like:

```
method: GET
route: /api/orders
status: 200
latency: 120ms
timestamp: ...
```

The monitoring server receives this data, does some basic preprocessing such as validating the metric or adding service information, and then stores it in PostgreSQL.

This design is simple and easy to understand. There is no queue or additional processing layer involved.

But there is an obvious problem. The monitoring request becomes part of the original request flow. If the application waits for the monitoring server to receive or process the metric, every API request gets some additional latency.

This is exactly what we don't want. A monitoring system should help us understand the application, not slow the application down.

So the next question is: **how do we collect these metrics without making the host application wait for the monitoring system?**

Making Metric Collection Asynchronous

The first problem with our simple design is the extra latency added to the host application.

So we change one thing: ****the application should never wait for the monitoring system.****

We use the Express `finish`` event to trigger metric collection only after the response has already been sent to the client.

```text

Client



Backend



Response sent



```

The SDK collects the request information and makes a non-blocking HTTP `POST`` request to the monitoring server using `fetch``. We don't await this request, so the application doesn't have to wait for the monitoring server before completing the original request.

This gives us a much better property: ****if the monitoring server is slow or unavailable, the host application can still continue serving requests.****

There is a trade-off here, though. We don't keep a local queue or retry failed requests in the SDK. If the monitoring server is unavailable, that metric is simply dropped.

For a monitoring system, this is an acceptable trade-off for now. We care more about making sure the monitoring layer never becomes a reason for the host application to slow down or crash.

But now we have another problem.

Every request still creates a separate HTTP call to our monitoring server. As the number of requests grows, the monitoring server has to receive and process a large number of metric events. If it also writes every event directly to PostgreSQL, it can quickly become the next bottleneck.

So we have solved the **latency problem**, but now we need to solve the **ingestion problem**.

The Ingestion Bottleneck

Now that metric collection is asynchronous, the host application doesn't have to wait for the monitoring system anymore.

But we still have another problem on the monitoring side.

Let's say our backend receives 1,000 requests per second. Since we collect one metric for every completed request, the monitoring server also receives around 1,000 metric requests per second.

If the monitoring server handles everything itself, the flow looks like this:

Monitoring Server



Receive metric



Process metric



Write to PostgreSQL



Send response

At a higher request rate, the monitoring server now has to receive, process, and store a large number of metrics at the same time. The database might be able to handle it, but we don't really want metric ingestion to be tightly coupled with metric processing.

What if the database becomes temporarily slow? Or the number of incoming metrics suddenly increases?

This is where we introduce a queue.

Instead of processing the metric immediately, the monitoring server publishes it to RabbitMQ and returns quickly.

Host Application



SDK



Monitoring Server



RabbitMQ



Workers



PostgreSQL

Now the Monitoring Server mainly handles ingestion, while the workers handle processing and storage.

For example, if 10,000 metrics arrive in a short period but the workers can process only 7,000 immediately, the remaining metrics can stay in RabbitMQ until workers are ready.

So RabbitMQ gives us a buffer between the fast incoming traffic and the slower processing side.

More importantly, we can now scale these two parts independently. If metric processing becomes the bottleneck, we can add more workers without changing how the SDK sends metrics.

We have now separated "receiving metrics" from "processing metrics".

But this also raises another question: why RabbitMQ specifically, and what do we get from using it?

Why RabbitMQ?

We use RabbitMQ mainly for three reasons: buffering, asynchronous processing, and scaling.

Instead of making the Monitoring Server directly write every incoming metric to PostgreSQL, we first put the metric into a queue.

Monitoring Server



RabbitMQ



Workers



PostgreSQL

RabbitMQ gives us a buffer when metrics arrive faster than the workers can process them.

It also keeps ingestion and processing separate. The Monitoring Server can quickly accept incoming metrics, while workers process them independently.

And as the traffic grows, we can add more workers to process messages in parallel instead of making the Monitoring Server handle everything itself.

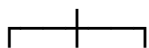
So RabbitMQ gives us a simple way to buffer the load, process metrics asynchronously, and scale the processing layer independently.

Worker Architecture

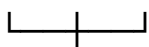
Once the metrics reach RabbitMQ, workers consume them from a shared queue.

We can run multiple worker instances, and RabbitMQ distributes the messages between them. This lets us scale the processing layer independently as the metric volume grows.

RabbitMQ



W1 W2 W3



PostgreSQL

Each worker identifies the metric type, processes the required fields, and stores it in PostgreSQL.

We use manual acknowledgements, so a message is acknowledged only after successful database insertion. If a worker crashes before that, RabbitMQ can deliver the message to another worker.

For temporary database failures, we requeue the message. Invalid messages are dropped instead of being retried forever.

This gives us parallel processing with basic failure protection.

Database Design

Once the workers process the metrics, we need to store them somewhere. We use PostgreSQL for this.

For raw metrics, we mainly have two tables:

`request_metrics`

`system_metrics`

`request_metrics` stores information like the endpoint, method, status code, latency, and timestamp. `system_metrics` stores CPU and memory usage along with the timestamp.

We also add indexes based on how we query the data. For example, queries for a project over a time range use the `projectId` and `timestamp` fields, while endpoint-level queries use the `route`.

But there is another problem we need to think about.

A busy backend can generate millions of request metrics over time. Running **COUNT**, **AVG**, or percentile calculations directly on millions of raw rows every time the dashboard is opened is not something we want.

So we keep the raw metrics for recent data and introduce pre-aggregated time buckets for larger time ranges.

Raw Metrics



1-Minute Aggregates



1-Hour Aggregates



1-Day Aggregates

This lets us query a much smaller amount of data when looking at longer time periods.

Alerting System

Now that we have the metrics stored, we also want the system to tell us when something starts going wrong.

We define alert rules based on metrics like **errorRate**, **avgLatencyMs**, **maxLatencyMs**, CPU usage, memory usage, or request count.

For example:

avgLatencyMs > 500

window: 60 seconds

Every minute, a background job checks these rules over their configured time window. It calculates the required metric and compares it with the threshold.

If the threshold is breached, we send an alert to Slack.

But we don't want to receive the same alert every minute while the issue is still happening. So each rule keeps a **triggered** state. We send an alert only when the state changes from not triggered → triggered.

Once the metric goes back to normal, the state changes back and we send a single resolved notification.

Normal



Threshold breached



 Alert



Still breached → No new alert



Back to normal



 Resolved

This gives us alerts that are useful without flooding Slack with the same message repeatedly.

Failure Handling

Once multiple components are involved, we also need to think about what happens when one of them fails.

If RabbitMQ is down, the Monitoring Server can't queue the metric and returns a **500**. The SDK catches this and silently drops the metric.

If a worker crashes, any unacknowledged messages remain in RabbitMQ and are delivered to another worker. Similarly, if PostgreSQL is temporarily unavailable, the worker requeues the message instead of losing it.

The Monitoring Server itself can also go down. In that case, the SDK simply drops the metric, while the host application continues running normally.

For Slack, a failed webhook doesn't affect the monitoring system, but the notification is lost for that evaluation cycle.

So we intentionally prioritize the host application's availability. Losing a monitoring metric is acceptable in some failure cases, but we don't want the monitoring system to become a dependency that can take down the application.

Final Architecture

After solving the problems one by one, our final architecture looks like this:

Host Applications



SDK



Load Balancer



Monitoring API



RabbitMQ



Multiple Workers



PostgreSQL



Aggregation / Alerts



Dashboard / Slack

The SDK runs inside the host application and collects request and system metrics. It sends these metrics asynchronously to the Monitoring API, so the application doesn't have to wait for the monitoring system.

The Monitoring API stays stateless, so we can run multiple instances behind a load balancer. Its main job is to accept metrics and publish them to RabbitMQ.

RabbitMQ acts as the buffer between ingestion and processing. Multiple workers consume messages from the queue and store the processed metrics in PostgreSQL.

PostgreSQL stores both raw and aggregated metrics. The monitoring system uses this data for dashboard queries and for evaluating alert rules.

Finally, the dashboard provides a way to inspect the metrics, while the alerting system sends notifications to Slack when configured thresholds are crossed.

The important part of this architecture is that each layer has a clear responsibility. More importantly, the layers that handle incoming traffic and the layers that process metrics can scale independently.

Design Trade-offs & Limitations

No system design is perfect. We make trade-offs based on our current scale and requirements.

1. Metric Loss

If the Monitoring API or RabbitMQ is unavailable, the SDK drops the metric instead of retrying indefinitely. We prioritize application availability over complete telemetry.

2. SDK Batching

Currently, the SDK sends one asynchronous request per metric. At higher scale, we can add batching and sampling to reduce network overhead.

3. PostgreSQL

PostgreSQL keeps the system simple, but very large metric volumes may require a dedicated time-series database such as TimescaleDB or ClickHouse.

4. Slack Alerts

Slack notifications are fire-and-forget, so a failed notification can be lost. A retry queue can solve this later.

5. Alert Evaluation

Alerts are checked every 60 seconds using `node-cron`, which keeps things simple but adds some detection delay.

6. RabbitMQ

We currently use a single RabbitMQ broker, which is simple but creates a single point of failure. A highly available setup can be added later.

7. Service Registration

Workers use a database upsert to register services automatically. At higher scale, this can be cached to reduce database operations.

The current design favors simplicity, isolation, and scalability where it matters, while leaving clear paths for future improvements.

Scaling 10×

Let's say the system now needs to handle around 10× the traffic of our current setup.

We don't need to completely redesign the architecture. We can scale the existing components and remove the biggest sources of unnecessary overhead.

1. Batch metrics in the SDK

The first improvement is on the SDK side.

Instead of sending one HTTP request for every metric, the SDK can keep a small in-memory buffer and send metrics in batches.

Before:

Request → POST /collect

Request → POST /collect

Request → POST /collect

After:

Request —┐

Request —|

Request —| → Buffer → Batch POST /collect

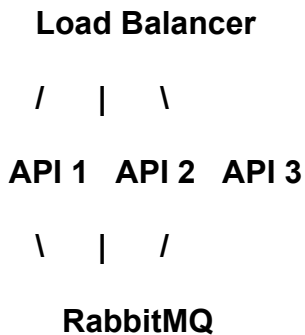
Request —|

For example, the SDK can flush when the buffer reaches 100 metrics or after a few seconds.

This significantly reduces the number of HTTP requests going to the Monitoring API.

2. Add more Monitoring API instances

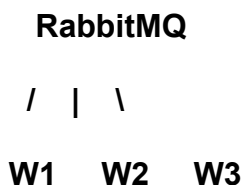
The Monitoring API is stateless, so we can simply add more instances behind the load balancer.



The SDK doesn't need to know how many instances are running. The load balancer distributes the incoming batches between them.

3. Add more workers

As the number of metrics increases, we can also increase the number of worker instances.



W4 W5 W6



PostgreSQL

Since the workers consume from the same queue, RabbitMQ distributes the messages between them.

This lets us increase processing capacity without changing the ingestion layer.

4. Improve PostgreSQL

At 10× scale, PostgreSQL starts handling significantly more metric data.

Before immediately replacing it, we can first improve how we use it:

- Keep the existing indexes aligned with dashboard queries.
- Continue using minute/hour/day aggregates so dashboards don't scan large amounts of raw data.
- Remove old raw metrics after the retention period.
- Add read replicas if dashboard queries start competing with metric writes.

This lets us keep PostgreSQL while handling more load.

So our 10× architecture still looks very similar:

Host Applications



SDK + Batching

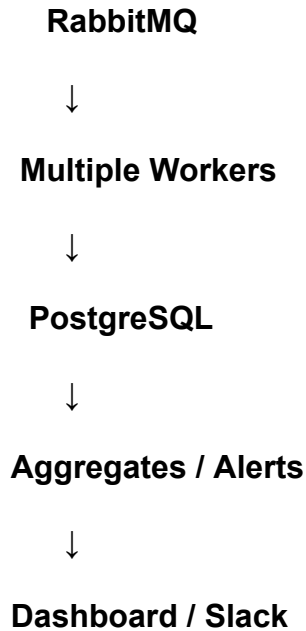


Load Balancer



Multiple Monitoring APIs





The main idea is simple: we don't redesign everything just because traffic increases. We first remove unnecessary network overhead, add more instances where the system is stateless, increase worker capacity, and optimize the database.

Conclusion

The biggest thing we take away from building this system is that designing a scalable system doesn't mean starting with a complex architecture.

We start with the simplest design that solves the problem:

Backend → Monitoring Server → Database

Then we ask what can go wrong.

The monitoring request adds latency, so we make metric collection asynchronous.

The Monitoring API has to handle both ingestion and processing, so we introduce RabbitMQ and separate ingestion from processing.

The number of metrics grows, so we add multiple workers and let them process messages in parallel.

The amount of data grows, so we introduce indexes and pre-aggregated metrics.

Finally, when traffic grows further, we scale the stateless API layer, workers, and SDK using batching.

This way of thinking is what makes the system scalable. We don't add Kafka, multiple databases, or other complex components just because they are common in large systems. We introduce a new component when the current design creates a problem that needs to be solved.

For us, the most important lesson is simple:

Start simple. Find the bottleneck. Solve it. Then find the next bottleneck.

That approach makes system design much easier to reason about and also helps us understand why each component exists in the final architecture.